

* Assembler Directives:-

- These are not instructions to 8051 mc.
- These are actually instⁿ to assembler and directs assembler program what to do during process of assembling.
- The assembly directives do not have any effect on content of 8051 mc memory (except DB & DW directives).

* ① DB (Define Byte):- It is used to define 8-bit data. The data can be in decimal, binary, hex or ASCII format.

→ For Decimal 'D' after number is written or optional, but for binary 'B' is written, for Hexadecimal 'H' is required.

→ Regardless of which format is used, assembler will convert numbers into Hex.

→ For ASCII characters single quote is used. e.g. 'A'.

→ This directive is only directive that can be used to define ASCII strings larger than two characters.

e.g. DB 10

DB 30H

DB 'STR'

DB 101101B

② DW (Define Word):- It is used to define 16-bit data. Its working is same as of DB directive except it is used for 16-bit values.

e.g. DW 1050H.

③ BIT:- It defines a bit constant, which is stored in bit addressable section of internal RAM. BIT directive is widely used to assign

bit-addressable I/O & RAM Locations. It allows program to assign the I/O or RAM bit at beginning of program, making it easier to modify them.

e.g. LED BIT P2.1

*④ ORG :- (Origin) → It is used to indicate the beginning or starting address of program in code memory.

e.g. ORG 0101H

*⑤ EQU (Equate) :- This directive is used to define a constant without occupying a memory locⁿ.
→ This directive does not set a side storage for a data item but associates a constant value with a data label appears in program, its constant value will be substituted for label.

e.g. COUNT EQU 25
MOV R3, #COUNT

→ Advantage of EQU directive is that, programmer can change it once & assembler will change all of its occurrences, rather than search the entire program trying to find every occurrence.

*⑥ END :- This indicates to assembler end of source (.asm) file or end of program.

→ End directive is last line of 8051 program i.e. source code, anything after END directive is ignored by assembler.

* 8051 Datatypes:-

- 8051 uic has only one data type-8-bit
- 8051 uic has 8-bit register, so programmer must break down larger sized data in 8-bit data.
- data type can be positive or negative
- + → Data in register ~~has~~ can be in following formats.

① Decimal:-

- As 8051 has 8-bit register, it can store value from 0 to 255 for unsigned int. & for signed integer values from -127 to ~~128~~ +128.

② Binary - we can store data in binary format i.e. in '0' & '1' in register by using it's single bit.

③ Hex:-

- As 8051 has 8-bit register, it can store values from 00H to FFH.
- For Hex number, number must start with digit, & number starts with alphabet is not recognised by uic.
e.g. 0A0H

* 8051 Addressing Modes:-

- The CPU can access data in various ways. The data could be in a register or in memory or may be immediate value. These various ways of accessing data are called addressing modes.

→ An addressing mode refers to how you are addressing a given memory location or way to address an operand.

* Types of Addressing modes:-

→ 8051 provides total five distinct addressing modes as follows:-

- ① Immediate
- ② Register
- ③ Direct
- ④ Indirect / Register Indirect
- ⑤ Index

① Immediate addressing mode:-

→ In this mode source operand is constant. In this mode as name implies, when instruction is assembled, the operand comes immediately after opcode.

→ The immediate data must be preceded by hash tag or pound sign '#'. This addressing mode can be used to load information into any of the registers, including DPTR regi.

e.g. `MOV A, #15H`

`MOV Rn, #28H`

`MOV DPTR, #2000H`

Here Rn is Regi. from R₀ to R₇ i.e. n = 0 to 7.

- Although DPTR is 16-bit, it can be accessed as two 8-bit registers as DPH & DPL.

e.g. `MOV DPL, #50H`

`MOV DPH, #40H`

instead of `MOV DPTR, #4050H`

→ We can also use immediate addressing mode to send data to 8051 ports.

② Register addressing mode:-

→ It involves the use of registers to hold data to be manipulated.

→ In this operand is placed in one of 8-bit or 16-bit general purpose registers.

e.g. `MOV A, R0`
`MOV R2, A`

* → Here source & destination register must match in size, i.e. `MOV DPTR, A` will give an error. Since source is 8-bit & destination is 16-bit register.

* → Programmer can move data between Acc to R_n ($n=0$ to 7), but can not move R_n to R_n by writting register name at both operand i.e. `MOV R3, R4` is invalid.

* → To move data betⁿ regi. one Regi. name & other must be memory locⁿ of that register or both regi. memory locⁿ/Address is valid i.e.

`MOV R3, 04`

`MOV 03, 04`

`MOV 03, R4` are valid instⁿ.

③ Direct Addressing mode:-

→ In this mode, the source or destⁿ address is specified by using 8-bit data in instⁿ. only internal data memory can be used in this mode.

→ In this mode, the data is in RAM locⁿ whose address is known & this address is given as a part of instⁿ.

e.g. `MOV R0, 40H`

`MOV 56H, A`; moves contents of A in RAM Locⁿ 56H.

* → The address value is limited to one byte from 00 to FFH, which means this addressing mode is limited to accessing RAM locⁿ & Regi. inside 8051.

* → Only direct addressing mode is allowed for pushing onto stack i.e. `PUSH 0F0H` is valid & `PUSH A` is invalid instⁿ.

* → Same as PUSH instⁿ POP is used i.e. POP 05, here POP instⁿ pops top of stack in R5 of bank 0.

④ Indirect / Register Indirect addressing mode:-

→ In this mode, a register is used as a pointer to data. If data is inside CPU only registers R0 & R1 are used.

→ R2 to R7 can not be used to hold address of an operand located in RAM, when using this addressing mode.

→ When R0 & R1 are used as pointer i.e. when they hold address of RAM Locⁿ, they must be preceded by '@' sign.

e.g. `MOV A, @R0`
`MOV @R1, A`

→ In this mode only R0 & R1 Regi. can be used as pointers. Since R0 & R1 are 8-bit wide their use is limited to accessing any information in internal RAM (00H to 7FH or SFR).

→ For accessing externally connected RAM or on-chip ROM, we need 16-bit pointer, DPTR is used

e.g. `MOVX A, @DPTR`

⑤ Index Addressing Mode:- used

→ This mode for accessing data elements in look-up table located in ROM of 8051.

→ The instⁿ used for this purpose is
" `MOVC A, @A+DPTR`

Here `MOVC` represents `MOV` code memory.

- 16-bit regi. DPTR & regi. A are used to form address of data element stored in on-chip ROM.